

JavaScript Fundamentals

Lecture 1: Introduction to the Language of the Web

A comprehensive deep dive into variables, data types, control flow, and functions — the building blocks of every modern web application.

DURATION

80–90 Mins

LEVEL

Beginner /
Intermediate

hello.js

```
console.log('Hello, JavaScript!');  
const sum = 10 + 20;  
console.log(`Sum is: ${sum}`);
```

► **CONSOLE OUTPUT**

```
$ node hello.js  
Hello, JavaScript!  
Sum is: 30
```

AGENDA

Lesson Roadmap

What we will cover today — from zero to writing real JavaScript programs.

01

What is JavaScript?

where it runs, why it dominates the web

03

Variables & Constants

let, const, var

05

Operators & Comparisons

arithmetic, ===, logical

07

Type Coercion

the famous JS quirks

09

Functions

declarations, expressions, arrows

02

Running JS

browser console, Node.js, and the REPL

04

Data Types & typeof

primitives vs objects

06

Strings & Template Literals

modern text handling

08

Control Flow & Loops

if / else, switch, for, while

10

Arrays & Objects

plus map / filter / reduce

FOUNDATION

What is JavaScript?

The language that powers the interactive web — and, with Node.js, the server too.

- **Created in 1995** — by Brendan Eich at Netscape — built in just 10 days.
- **Runs everywhere** — in every browser (frontend) and on servers via Node.js (backend).
- **High-level & dynamically typed** — no manual compilation, types checked at runtime.
- **Multi-paradigm** — object-oriented + functional styles both work naturally.
- **NOT Java!** — "Java is to JavaScript as car is to carpet" — the name was pure marketing.

⚡ The 3 Core Web Technologies

HTML = structure, CSS = style, JavaScript = behavior. Every interactive page you have used runs on these three.

WHERE DOES JS RUN?

Browser

Chrome (V8) · Firefox (SpiderMonkey) · Safari (JavaScriptCore)
frontend logic, DOM, UI

Server

Node.js runtime — our focus in this course
APIs, file system, backend logic

Tools & Apps

VS Code extensions, Electron, Photoshop scripting
JS is a general-purpose language now

FOUNDATION

Running JavaScript

Two ways to execute code — experiment in the browser, ship it with Node.js.

1

In the Browser

Open any page, press F12 → Console tab, type JavaScript, press Enter. Instant feedback, zero setup.

2

With Node.js

Save your code in a .js file and run it from the terminal: `node filename.js`. This is how real programs ship.

✓ The REPL

Type **node** in your terminal for a Read-Eval-Print Loop — instant snippet testing.

► INTERACTIVE REPL

```
$ node
Welcome to Node.js v22.23.2.
Type ".help" for more information.
> 1 + 2
3

> 'Hello' + ' ' + 'World'
'Hello World'

> const lang = "JS"
undefined
> lang + " rocks"
'JS rocks'

> .exit
```

CORE CONCEPT

Variables: let, const, var

Three ways to declare state — one rule of thumb: const by default.

variables.js

```
// let -> can be reassigned
let score = 10;
score = score + 5;
console.log('score:', score);

// const -> cannot be reassigned
const name = 'Ada';
console.log('name:', name);

// var -> legacy, avoid
var old = 'still works';
console.log('var:', old);
```

► CONSOLE OUTPUT

```
$ node variables.js
score: 15
name: Ada
var: still works
```

THE RULES ▪ **const** — default choice — binding cannot be reassigned.

- **let** — only when the value must change (counters, loops).
- **var** — legacy, function-scoped, error-prone — avoid it.

⚠ Reassigning a const

`pi = 3.14159;` → crashes immediately: **TypeError: Assignment to constant variable.**

⚡ **const ≠ immutable value**

const prevents reassignment, not mutation — arrays and objects declared with const can still change their contents.

CORE CONCEPT

Data Types & typeof

7 primitive types + objects. Use `typeof` to inspect any value at runtime.

types.js

```
const text      = 'JS Rocks';    // String
const age       = 25;            // Number
const isActive  = true;          // Boolean
const nothing   = null;          // null (empty)
let notSet;                    // undefined
const big       = 9007199254741999n; // BigInt
const unique    = Symbol('id');  // Symbol

const skills = ['HTML', 'CSS', 'JS'];
const user   = { name: 'Ada' };

console.log(typeof text, typeof age, typeof isActive);
console.log(typeof nothing, typeof notSet, typeof big);
console.log(typeof skills, typeof user);
```

► CONSOLE OUTPUT

```
$ node types.js
string number boolean
object undefined bigint
object object
```

QUICK REFERENCE

String	'JS Rocks'
Number	25, 3.14, NaN, Infinity
Boolean	true / false
null	intentional "empty"
undefined	declared, not assigned
BigInt	9007199254741999n
Symbol	unique identifiers
object	arrays, objects, functions...

⚠ Famous JS quirk

typeof null returns **'object'** — a 30-year-old bug kept for backwards compatibility. Check null with **value === null** instead.

CORE CONCEPT

Operators & Comparisons

Arithmetic, comparison and logical operators — and the == vs === trap.

operators.js

```
// Arithmetic
console.log(10 + 3, 10 - 3, 10 * 3,
            10 / 3, 10 % 3, 2 ** 3);

// == converts types, === does not
console.log(5 == '5'); // loose
console.log(5 === '5'); // strict
console.log(5 !== '5');

// Logical
const isAdult = true, hasTicket = false;
console.log(isAdult && hasTicket); // AND
console.log(isAdult || hasTicket); // OR
console.log(!isAdult);           // NOT
```

► CONSOLE OUTPUT

```
$ node operators.js
13 7 30 3.3333333333333335 1 8
true
false
true
false
```

OPERATOR CHEAT SHEET

+ **-** ***** **/** **%** math; % = remainder, ** = power

== **!=** loose: converts types first

=== **!==** strict: value AND type ✓

> **<** **>=** **<=** numeric / string ordering

&& **||** **!** logical AND / OR / NOT

+= **-=** **++** **--** shorthand assignment & increment

✓ Best practice

Always use **===** and **!==**. Loose **==** does surprise type conversions ('5' == 5 is true) and is a common source of bugs.

CORE CONCEPT

Strings & Template Literals

Text handling — backticks + `${}` replace messy concatenation forever.

strings.js

```
const name = 'Ada', lang = 'JavaScript';

// Old way: concatenation
console.log('Old: ' + name + ' learns ' + lang);

// Modern way: template literals
console.log(`New: ${name} learns ${lang}`);

const course = ' Web Technologies ';
console.log(course.length);
console.log(course.trim());
console.log(course.trim().toUpperCase());
console.log(lang[0], lang.slice(0, 4));
console.log(lang.includes('Script'));
```

► CONSOLE OUTPUT

```
$ node strings.js
Old: Ada learns JavaScript
New: Ada learns JavaScript
20
Web Technologies
WEB TECHNOLOGIES
```

WHY TEMPLATES WIN ▪ **Interpolation** — embed any expression: ``Total: ${price * 1.2}``

- **Multi-line** — backtick strings span lines without `"\n"`.
- **Readable** — no more `" + " + "` chains.

⚡ Remember

Strings are zero-indexed: `lang[0]` is `'J'` — and `.length` counts every character, spaces included.

✓ Method chaining

`' Web Technologies '.trim().toUpperCase()` — methods return new strings you can keep calling on.

CORE CONCEPT

Type Coercion: The Famous JS Quirks

JavaScript converts types silently — know the rules before they bite you.

coercion.js

```
console.log('5' + 3);    // concat wins
console.log('5' - 3);    // forces numeric
console.log('5' * '2');
console.log(1 + true);   // true -> 1
console.log('' + 0);

// Falsy: false, 0, '', null, undefined, NaN
console.log(Boolean(0), Boolean(''),
             Boolean('hi'));
console.log(Boolean(null),
             Boolean(undefined), Boolean([]));
```

► CONSOLE OUTPUT

```
$ node coercion.js
53
2
10
2
0
```

⚠ FLOATING-POINT TRAP

```
console.log(0.1 + 0.2);
0.30000000000000004
```

Numbers are 64-bit floats. Never compare money with === — round it, or use integer cents.

THE 6 FALSY VALUES

false 0 "" null undefined NaN

Everything else is truthy — including "0", [] and {}!

This powers short ifs like: if (userInput) { ... }

WORKFLOW

Control Flow: if / else, Ternary & switch

Decide which code runs — the backbone of every program.

conditionals.js

```
const score = 85;

if (score >= 90) {
  console.log('Grade: A');
} else if (score >= 80) {
  console.log('Grade: B'); // runs
} else {
  console.log('Grade: C');
}

// Ternary: short if/else
const access = score >= 50
  ? 'granted' : 'denied';
console.log('Access:', access);

// switch
const day = 'sat';
```

► CONSOLE OUTPUT

```
$ node conditionals.js
Grade: B
Access: granted
Weekend!
```

1

if / else if / else

checks top → down; first true branch wins, rest skipped.

2

Ternary a ? b : c

an expression version of if/else — perfect for assigning one of two values.

3

switch

cleaner than long else-if chains; fall-through cases share one body.

4

default

the safety net when nothing matches — like a final else.

⚡ Watch fall-through

Without **break**, execution continues into the next case!

WORKFLOW

Loops

Repeat work without repeating code — four patterns you will use daily.

loops.js

```
// for: known count
for (let i = 1; i <= 3; i++) {
  console.log('Iteration', i);
}

// while: condition-driven
let battery = 100;
while (battery > 0) {
  battery -= 50;
  console.log('battery:', battery);
}

// for...of: walk an array
for (const lang of ['JS', 'Python', 'Go']) {
  console.log('I know', lang);
}

// break & continue
for (let i = 0; i < 5; i++) {
  if (i === 2) continue; // skip
  if (i === 4) break;    // stop
  console.log('i =', i);
}
```

► CONSOLE OUTPUT

```
$ node loops.js
Iteration 1
Iteration 2
Iteration 3
battery: 50
battery: 0
I know JS
I know Python
I know Go
i = 0
i = 1
i = 3
```

PICKING THE RIGHT LOOP ▪ **for** — you know how many times (counter).

- **while** — repeat until a condition flips.
- **for...of** — walk through an array or string.
- **break / continue** — exit early / skip one round.

CORE CONCEPT

Functions: Three Ways to Write Them

Reusable blocks of logic — arrows are the modern default.

functions.js

```
// 1. Function declaration
function add(a, b) {
  return a + b;
}

// 2. Function expression
const multiply = function (a, b) {
  return a * b;
};

// 3. Arrow function
const square = (n) => n * n;

// 4. Default parameter
const greet = (name = 'guest') =>
  `Hello, ${name}!`;

console.log(add(2, 3));
console.log(multiply(4, 5));
console.log(square(7));
```

► CONSOLE OUTPUT

```
$ node functions.js
```

```
5
```

```
20
```

ANATOMY OF A FUNCTION

- **parameters** — inputs in parentheses: (a, b)
- **return** — sends a value back; code after it never runs
- **call** — run it with (): add(2, 3) → 5
- **default param** — name = 'guest' used when no argument passed

⚡ Implicit return

Arrow functions with a single expression need no curly braces and no **return**: `n => n * n`.

✓ Style tip

Use declarations for named utilities, arrows for short callbacks.

DATA STRUCTURES

Arrays

Ordered lists of values — index starts at 0, always.

arrays.js

```
const langs = ['JS', 'Python', 'Go'];

// Access & length (index starts at 0)
console.log(langs[0], langs[2], langs.length);

// Modify
langs.push('Rust');           // add to end
console.log(langs);

const nums = [3, 1, 4, 1, 5];
console.log(nums.indexOf(4));
nums.sort(); console.log(nums);
console.log(nums.includes(9));
console.log(nums.slice(1, 3)); // no mutation
console.log(nums.pop(), nums); // remove last
```

► CONSOLE OUTPUT

```
$ node arrays.js
JS Go 3
[ 'JS', 'Python', 'Go', 'Rust' ]
```

METHOD CHEAT SHEET

- **push** / **pop** — add / remove at the end
- **shift** / **unshift** — remove / add at the start
- **indexOf** / **includes** — find an element
- **slice** — copy a section — original untouched
- **sort** / **reverse** — mutate the array in place!
- **join** — glue to a string: ['a','b'].join('-')

⚠ Mutation warning

sort, **reverse** and **splice** change the original — prefer non-mutating methods.

✓ Zero-based

An array of 3 has indexes 0, 1, 2 — **langs[3]** is **undefined**.

Objects: Key-Value Pairs

Group related data and behavior into one entity — the core of JS.

objects.js

```
const user = {  
  name: 'Ada',  
  age: 36,  
  greet() {  
    return `Hi, I am ${this.name}`;  
  }  
};  
  
console.log(user.name);    // dot  
console.log(user['age']);  // bracket  
console.log(user.greet());  
  
user.age = 37;              // update  
user.email = 'ada@calc.dev'; // add  
console.log(user);  
console.log(Object.keys(user));
```

► CONSOLE OUTPUT

```
$ node objects.js
```

```
Ada
```

KEY IDEAS ▪ **Keys + values** — strings → anything: numbers, arrays, even functions ('methods').

- **Dot vs bracket** — `user.name` is cleaner; `user["name"]` works with dynamic keys.
- **this** — inside a method, refers to the object itself.
- **const object** — binding is fixed, properties are not — `user.age = 37` is fine.

⚡ Shorthand

`{ name }` is short for `{ name: name }`. And `Object.keys/values/entries` loop over objects without `for...in`.

Destructuring & Spread

Unpack data with style — ES6+ features you will see in every modern codebase.

destructuring.js

```
const user = { name: 'Ada', age: 36,
               city: 'London' };
const langs = ['JS', 'Python', 'Go'];

// Object destructuring (+ default)
const { name, age, country = 'UK' } = user;
console.log(name, age, country);

// Array destructuring (skip middle)
const [first, , third] = langs;
console.log(first, third);

// Rest & spread
const { city, ...rest } = user;
console.log(city, rest);
console.log([...langs, 'Rust']);
```

► CONSOLE OUTPUT

```
$ node destructuring.js
```

```
Ada 36 UK
```

```
JS Go
```

DECODE THE SYNTAX ▪ **{ name } = obj** — pulls properties into one line.

- **country = 'UK'** — default value when the key is missing.
- **[first, , third]** — commas skip positions in arrays.
- **...rest** — gathers the leftovers into a new object.
- **[...langs, item]** — spread: a new array with extras (no mutation).

✓ Why it matters

Destructuring is everywhere in real code — function parameters, React props, API responses. Learn it now.

Higher-Order Functions: map · filter · reduce

Replace most loops with three expressive array methods.

hof.js

```
const prices = [120, 80, 45, 200];

// map -> transform every element
const withTax = prices.map(p => p * 1.2);
console.log('map   :', withTax);

// filter -> keep matching elements
const cheap = prices.filter(p => p < 100);
console.log('filter:', cheap);

// reduce -> boil down to one value
const total = prices.reduce((sum, p) => sum + p, 0);
console.log('reduce:', total);

// chaining
console.log('chain :',
  prices.filter(p => p >= 100)
    .map(p => p / 2));
```

► CONSOLE OUTPUT

```
$ node hof.js
map   : [ 144, 96, 54, 240 ]
filter: [ 80, 45 ]
```

map

transform every element, same length out
`prices.map(p => p * 1.2)`

filter

keep only elements that pass the test
`prices.filter(p => p < 100)`

reduce

accumulate everything into one value
`reduce((sum, p) => sum + p, 0)`

✓ Learn them by heart

They replace most loops in modern code — everywhere in React, Node and real APIs.

PRACTICE

Hands-On Lab

Create a scratch folder and run every snippet with `node filename.js` — typing beats copy-paste.

1

Warm-up

Store name and age in `const`, print "Ada is 36 years old" using a template literal.

2

FizzBuzz Lite

Loop 1..15. Print "Fizz" for /3, "Buzz" for /5, "FizzBuzz" for both.

3

Cart Total

`const cart = [120, 80, 45];` compute total with `reduce`, then 20% off with `map`.

4

Object Play

Create a book with `title/author/year`, add a `summary()` method, print with dot notation.

5

Type Detective

Predict `console.log('5' + 3, '5' - 3, Boolean(''))` FIRST, then verify with node.



Tip

Run each exercise as its own file — `node warmup.js` — so failures never hide each other.

SUMMARY

Recap & Next Steps

What you mastered today, and what to practice before next week.

✓ **KEY TAKEAWAYS** ▪ JS runs in the browser and on servers (Node.js) — one language everywhere.

- **const by default** — let when needed, avoid var.
- 7 primitive types + objects/arrays — inspect with typeof.
- **Always ===** — beware coercion and the 0.1 + 0.2 trap.
- **Functions × 3 flavors** — arrows are the modern default.
- **map / filter / reduce** — the workhorses of array processing.

△ **PRACTICE ASSIGNMENTS** ▪ Re-run every snippet from today in the REPL — predict outputs first.

- Build a number-guessing game using while + if/else.
- Rewrite one for loop as map + filter.
- Read: MDN JavaScript Guide · javascript.info

COMING NEXT LESSON

→ **Node.js Fundamentals: Runtime, Modules & Your First HTTP Server**